

Présentation du projet et règles de travail

Vous allez réaliser, de la première ligne à la livraison, l'intégration du site du festival Résonances. Le projet commence par la construction d'une bibliothèque de composants réutilisables ; les pages du site sont ensuite assemblées à partir de ces composants. Chaque étape reprend le travail de la précédente : à la fin du module, vous disposerez d'un site complet, d'une feuille de style organisée et d'un guide de style documenté.

Le contexte

Le festival Résonances se tient chaque été pendant trois jours au bord d'un lac. Il accueille une quarantaine d'artistes répartis sur trois scènes, chacune avec son identité visuelle.

Scène	Programmation	Couleur dominante
Scène du Lac	Pop et rock	Bleu profond
Scène de la Forêt	Musiques électroniques	Vert sapin
Le Kiosque	Jazz et acoustique	Ocre

L'association organisatrice vous confie l'intégration du site. Le graphiste a livré une maquette de cinq pages, accompagnée d'un guide visuel qui relève les couleurs, la typographie et les espacements, et d'une planche « 06 · Composants » qui présente la bibliothèque de composants à réaliser. Votre rôle est d'écrire une feuille de style qui reste maintenable lorsque d'autres développeurs reprendront le projet l'année suivante.

Maquette Figma : <https://www.figma.com/design/5mlrycFNyrEHFl6nn9OzhW>

Les pages à réaliser

1. **Accueil** : bannière principale, présentation du festival, têtes d’affiche, accès aux scènes.
2. **Programme** : concerts classés par jour, filtrables par scène.
3. **Fiche artiste** : photo, biographie, créneau de passage, autres artistes de la même scène.
4. **Billetterie** : trois formules de pass et un formulaire de réservation.
5. **Infos pratiques** : accès, horaires, questions fréquentes.

S’y ajoute une sixième page, [composants.html](#), qui rassemble tous les composants du site avec leurs variantes et leurs états. Sa maquette correspond au cadre « 06 · Composants » du fichier Figma.

La démarche

Le projet suit cet ordre, en trois temps :

1. **Mettre en place l’environnement** (étapes 1 et 2) : compilation Sass, architecture SMACSS, jetons de conception.
2. **Construire la bibliothèque de composants** (étapes 3 à 6) : boutons, badges, cartes, formulaires, puis factorisation avec les outils de Sass.
3. **Assembler et enrichir le site** (étapes 7 à 10) : intégration des pages, thèmes, fonctionnalités, livraison.

La page [composants.html](#) est le fil conducteur du projet : créée à l’étape 2, elle s’enrichit à chaque étape et devient, à l’étape 10, le guide de style livré avec le site.

Objectifs pédagogiques

À l’issue du projet, vous serez capable de :

- concevoir des composants réutilisables, indépendants de la page où ils sont placés, avec leurs variantes et leurs états ;
- organiser une feuille de style selon l’architecture SMACSS ;
- nommer vos classes avec la convention BEM et documenter vos choix ;
- centraliser les valeurs de conception dans des variables Sass et des propriétés personnalisées, et décliner des thèmes ;
- structurer un projet Sass en partiels et le factoriser avec des mixins, des fonctions et des boucles ;
- livrer un fichier CSS unique, optimisé et documenté.

Déroulement

Le projet compte dix étapes. Chacune comporte au moins deux exercices.

Étape	Thème
1	Environnement Sass et architecture SMACSS
2	Jetons de conception et page des composants
3	Composants : boutons et badges
4	Composants : cartes
5	Composants : formulaires, filtres et états
6	Factoriser avec Sass : mixins, fonctions, boucles
7	Assemblage des pages
8	Thèmes des scènes et mode sombre
9	CSS moderne : @layer, color-mix(), @container
10	Livraison : optimisation, guide de style, recette

Règles techniques

Ces règles sont communes à tous. Elles portent sur la méthode et non sur les noms : le choix des noms de classes vous appartient, à condition de respecter la syntaxe BEM et de rester cohérent d'un bout à l'autre du projet.

Un seul fichier CSS

Les styles sont écrits exclusivement en SCSS, dans le dossier `scss/`. Le compilateur Sass produit un fichier unique, `css/style.css`, qui est le seul fichier de style relié aux pages. Aucun fichier `.css` n'est écrit à la main, et aucun style n'est placé dans un attribut `style` ou une balise `<style>`.

Organisation SMACSS

```
resonances/  
├ index.html, programme.html, artiste.html,  
| billetterie.html, infos.html, composants.html  
├ package.json  
├ NOMMAGE.md  
├ assets/img/  
├ scss/  
|   ├── abstracts/ variables, mixins et fonctions Sass  
|   ├── base/    reset, typographie, éléments HTML sans classe  
|   ├── layout/  en-tête, pied de page, grilles  
|   ├── modules/ composants, un partial par composant  
|   ├── state/   classes d'état  
|   ├── theme/   thèmes des scènes et mode sombre  
|   └─ main.scss point d'entrée  
└─ css/  
    └─ style.css fichier compilé, jamais modifié à la main
```

Le dossier **abstracts/** ne fait pas partie des cinq catégories SMACSS : il regroupe les outils propres à Sass, qui ne produisent aucune règle CSS par eux-mêmes.

Vous consignez vos conventions dans le fichier **NOMMAGE.md** : préfixes retenus, langue des noms, liste des blocs avec leurs éléments et leurs modificateurs. Ce fichier est tenu à jour à chaque étape et sert de référence lors de l'évaluation.

Règles d'écriture

- Aucun sélecteur d'identifiant : les **id** sont réservés au JavaScript et aux ancres.
- Un composant ne dépend jamais de son contexte : une variation s'exprime par un modificateur, pas par un sélecteur qui passe par un parent.
- L'imbrication Sass ne dépasse pas trois niveaux ; elle sert surtout à écrire les éléments, les modificateurs et les pseudo-classes avec **&**.
- **! important** est interdit.
- Aucune couleur, taille ou espacement n'est écrit en dur dans les modules : ces valeurs proviennent des jetons définis à l'étape 2.
- Le JavaScript cible des attributs **data-***, jamais des classes de style. Vos noms de classes peuvent ainsi évoluer sans casser les scripts.
- Chaque partial commence par un commentaire qui indique son rôle.

Dépôt du travail et livrables

À la fin de chaque étape, vous devez déposer votre travail sur votre dépôt Git. Pour chaque dépôt, le message du commit doit obligatoirement indiquer le numéro de l'étape réalisée. Par exemple :

```
git commit -m "Étape 2 : création des composants"
```

Le livrable final doit contenir les éléments suivants :

- Le code source en **SCSS** ;
- Le fichier CSS compilé `css/style.css` ;
- Les six pages HTML ;
- Le fichier `NOMMAGE.md` ;
- Le fichier `README.md`.

Mise en ligne du site

Le site doit être mis en ligne dès la création du projet. Vous pouvez utiliser l'une des solutions suivantes :

- **Vercel** ;
- **GitHub Pages**.

L'URL publique du site doit être ajoutée dans le fichier `README.md`.

Exemple :

```
https://mon-projet.vercel.app
```

Vérifiez que l'URL fonctionne correctement et qu'elle permet d'accéder au site.

Date limite

Tout commit effectué après la date limite sera considéré comme un travail remis en retard et pourra être pénalisé.

La date et l'heure du commit faisant foi, veillez à déposer régulièrement votre travail et à ne pas attendre le dernier moment.